

Operating Systems Exercise 1

Threading & Synchronization

Course: Operating Systems - Reichman University

Due date: 12.5.2026 (submit via INDlgiuous; **no late submissions**)

Submission rules (read carefully)

- Work in **pairs** (exactly two students per team).
- Submission is made through the INDlgiuous system.
- For the theoretical part, submit a PDF file containing your answer. You **MUST** type your answer (do not scan a hand-written solution).
- For the programming part, submit a ZIP file containing:
 - Subdirectories (case sensitive): `PQ1/` , `PQ2/` , `PQ3/`
 - Each `PQx/` directory must contain:
 - the `.c` and `.h` files requested for that PQ
 - `tests/` (your test source files)
 - `build_and_run_tests.sh`
 - Your `build_and_run_tests.sh` must compile test executables into an `output/` directory (create it with `mkdir -p output`).
 - Do not submit compiled binaries.
You may omit `output/` from the ZIP, but **make sure your script creates it**.
 - For example,
 - Your code for PQ1 is at `PQ1/*`
 - Your tests for PQ1 are at `PQ1/tests/*` .
 - The executables should be built to `PQ1/output/*` . You must **not** submit `PQ1/output/*` , but your script must create it.
 - **Include your header files** in the submission.
 - We will run your script from inside each `PQx` directory using:
 - `cd PQ[x] && bash build_and_run_tests.sh`
 - For example for PQ1: `cd PQ1 && bash build_and_run_tests.sh`
 - Make sure your zip does **not** contain a directory at its root.
When you zip your files, do **not** zip the folder that contains everything - zip the files **inside** it.
(After extracting the zip, we should immediately see your PDF, `PQ1/` , `PQ2/` , `PQ3/` , and not a folder)
- All source code will be scanned using plagiarism and LLM-detection tools. If suspicious similarity is detected, the team(s) may be investigated for academic misconduct according to university policy.

What you are implementing

You will implement several synchronization mechanisms using only atomics and spinning/yielding.

Important: Do not change the function signatures listed below.

You may add private helper functions inside `.c` files.

You may also add additional helper `.c/.h` files if needed (make sure your `build_and_run_tests.sh` compiles them if needed).

Part A - Theory (submit as a separate PDF)

Submit answers in a single PDF file.

Important: Use **clear, numbered headers** for each question (e.g. "TQ1", "TQ2", etc.) so the grader can locate your answers easily. Submissions with unclear or missing headers may be mis-graded.

TQ1 — Concurrency vs. parallelism

A program has 3 threads that each need 40_{ms} of CPU time. There is no I/O and no blocking.

1. On a single core, assuming the system switches between runnable threads and context-switch cost is negligible, what is the minimum wall-clock time until all threads finish?
2. On a 3-core CPU, same assumptions, what is the minimum wall-clock time?
3. If each context switch costs 0.2_{ms} , and on the single core the system forces a context switch after every 5_{ms} of CPU execution of the currently running thread, what is the extra overhead added purely due to context switching until completion?
Assume a context switch also occurs after the final 5_{ms} chunk that completes the last thread.

TQ2 — Interleavings

Question.

Shared/global integer `x` starts at 0. Two threads, A and B, run concurrently the following code **once each**:

```
t = x
t = t + 1
x = t
```

Assume `t` is a **thread-local** variable (each thread has its own `t`).

1. What final values of `x` are possible?
2. Give an explicit interleaving run that leads to each possible final value you stated in (1).

TQ3 — TAS vs Ticket lock

Consider the following two locks:

- a **test-and-set (TAS) spinlock**
- a **ticket lock** as we learned in the lecture (each thread gets `my_ticket = fetch_add(ticket,1)` and waits for `cur_ticket == my_ticket`).

1. Which lock guarantees **FIFO acquisition order**? Prove it.

Hint: Identify a per-thread "position" value assigned on arrival (e.g., a ticket number), and argue that the lock can

be acquired only in increasing order of that value.

2. Which lock guarantees **bounded waiting** (no starvation), assuming every thread that acquires the lock eventually releases it? Prove it.
Hint: If a thread gets ticket k and the current ticket is c , how many threads can enter before it? Express this as a bound.
3. Give a short argument (one paragraph) why TAS can starve a thread.

TQ4 — Producer/Consumer

We have a bounded buffer (queue) of capacity CAP , shared by multiple producers and consumers. Bounded semaphore value is always in $[0, CAP]$; unbounded semaphore has no fixed upper bound.

We use three synchronization objects:

- `mutex` — mutual exclusion for the buffer
- `empty` — a **bounded semaphore** with value always in $[0, CAP]$, initialized to CAP (number of empty slots)
- `full` — a **bounded semaphore** with value always in $[0, CAP]$, initialized to 0 (number of full slots)

A student wrote the following high-level algorithm:

Producer (buggy version):

1. `acquire mutex`
2. `wait(empty)`
3. `insert one item into the buffer`
4. `signal(full)`
5. `release mutex`

Consumer (buggy version):

1. `acquire mutex`
2. `wait(full)`
3. `remove one item from the buffer`
4. `signal(empty)`
5. `release mutex`

1. Explain why this algorithm can deadlock. Your explanation must include a concrete execution scenario (who holds what and who is waiting for what).
2. Give the **minimal fix** (change as little as possible) to make the algorithm deadlock-free, and explain why your fix works.

Part B - Programming

Allowed / forbidden (Programming part)

This section applies to all programming questions (PQ1–PQ3).

Assumption: each `PQx/` directory is self-contained. If you reuse code from another PQ, copy the needed `.c/.h` files into the current `PQx/` directory and compile them there (do not include files using paths like `../PQ1/...`).

What counts as “implementation files” vs “test files”?

- **Implementation files:** everything you write under PQ1/ , PQ2/ , PQ3/ **except** the tests/ directory.
 - Example: `tl_semaphore.c/.h` , `cond_var.c/.h` , `rw_lock.c/.h` , and any helper `.c/.h` files you add in the PQ root.
- **Test files:** everything under PQx/tests/ (these files contain `main()` and are compiled into executables).

Allowed (implementation files)

- Standard C headers (e.g., `stdio.h` , `stdlib.h` , `stdint.h` , `stdbool.h` , `assert.h` , etc.).
- `#include <stdatomic.h>` for atomic operations.
- `sched_yield()` from `#include <sched.h>` while spinning.
- `malloc/free` from `stdlib.h` **is allowed** (unless a specific PQ explicitly forbids it).
- Using primitives you already implemented in this exercise (e.g., ticket lock / semaphore / condition variable), as long as you follow each PQ’s requirements.

Forbidden (implementation files)

- Any pthread synchronization primitives, including (but not limited to):
 - `pthread_mutex_*` , `pthread_cond_*` , `pthread_rwlock_*` , `pthread_barrier_*` , `sem_*`
- Any OS/library synchronization mechanism other than `stdatomic` + spinning/yielding (as taught in this part of the course).
- Changing any required function signatures.

threads note (tests vs implementation)

- You may use threads **only in your tests (including the submitted tests)** for creating threads and joining them (`pthread_create` , `pthread_join` , and obtaining thread IDs if needed).
- Your submitted **implementation files** (PQ code files, not test files) must not depend on pthread synchronization primitives.

Building and running tests

0) Compilation environment (what we will use to grade)

- Ubuntu 24.04 LTS
- GCC version 13
- C17 (`-std=c17`)

On your machine, the compiler executable might be called `gcc` (common) or `gcc-13` . **Use whichever exists**, but make sure it is GCC 13.

If `gcc-13` exists use it, else use `gcc` (and verify `gcc --version` is 13).

Tip for bash scripts:

You can select the compiler like this:

```
CC=gcc; command -v gcc-13 >/dev/null && CC=gcc-13
```

and then compile using `$CC ...` .

To check your GCC version:

- `gcc --version`

If this prints version 13.x, you're good.

If it does not, try:

- `gcc-13 --version`

(Your `build_and_run_tests.sh` may use `gcc` or `gcc-13`. The important thing is that it compiles with GCC 13 on Ubuntu 24.04.)

1) Directory and build assumptions

For each PQ:

- The PQ root directory contains the implementation files (e.g., `tl_semaphore.c`, `tl_semaphore.h`, etc.).
- A subdirectory named `tests/` contains all test `.c` files.
- You must compile test executables into a subdirectory named `output/`.

Assumption: when compiling and running, your current working directory is always the **PQ root**.

2) Required script: `build_and_run_tests.sh`

For each PQ, you must submit a bash script named:

- `build_and_run_tests.sh`

This script must:

1. Compile all test executables into `output/`
2. Run all tests from `output/`
3. Exit with:
 - 0 if **all tests passed**
 - 1 if **at least one test failed** (or if compilation failed)

Your script must print which tests are running and whether they passed/failed. It is recommended you will print out additional logs and information to help you debug.

3) How tests include headers from the PQ root directory (important)

Your test files are inside `tests/`, but they need to include headers that are in the PQ root directory.

Do **not** write includes like:

- `#include "../tl_semaphore.h"` (avoid relative paths)

Instead:

- In the test file: `#include "tl_semaphore.h"`
- In the compile command, add the PQ root directory to the include search path using `-I`.

Example:

- `gcc ... -I. tests/test_unit.c tl_semaphore.c -o output/test_unit`

`-I.` tells gcc to look for headers in the current directory (the PQ root), so `#include "tl_semaphore.h"` works from any file.

4) Required compiler flags (enforced)

You must compile using **all** of the following:

- `-std=c17` (use C17)
- `-Wall -Wextra` (enable common warnings)
- `-Werror` (treat warnings as errors)
- `-pedantic` (enforce standard C rules more strictly)
- `-g` (include debug information; helps debugging)

So every compile command should look like:

- `gcc -std=c17 -Wall -Wextra -Werror -pedantic -g -I. ...`

Notes:

- We are enforcing warnings-as-errors because warnings in C often indicate real bugs.
- `-g` does not change program behavior; it just makes debugging easier.

5) Threading in tests

If a test uses pthreads (e.g., `pthread_create`, `pthread_join`), compile that test with:

- `-pthread`

Example:

- `gcc ... -pthread -I. tests/test_concurrent.c tl_semaphore.c -o output/test_concurrent`

6) Test executables and exit codes

Each test file should be compiled into its own executable under `output/`.

Example convention:

- `tests/test_unit.c` → `output/test_unit`
- `tests/test_concurrent.c` → `output/test_concurrent`
- `tests/test_stress.c` → `output/test_stress`

Each test program must return:

- `0` on success
- `1` on failure

If a test has multiple checks, it should return `1` if **any** check fails.

7) Common pitfalls

- If you see compilation errors about missing headers from tests, you probably forgot `-I.` or you used the wrong include.
- Do not rely on `printf` ordering between threads in stress tests — output can interleave. Use shared counters/flags for correctness checks.
- Your implementation files must not use pthread synchronization primitives (mutex/cond/semaphore/rwlock). Pthreads are allowed **only in tests** for creating/joining threads.

PQ1 — Unbounded semaphore with ticket lock

Implement an **unbounded semaphore** using the **ticket lock mechanism** presented in class. You must implement the functions declared in `tl_semaphore.h`.

Goal

Implement an unbounded semaphore that supports synchronization between multiple threads **without using any OS/library synchronization primitives** (no pthread mutex/cond/semaphore/rwlock, etc.).

When the semaphore value is 0, `semaphore_wait()` must **busy-wait** and call `sched_yield()` while waiting.

Unbounded semaphore semantics (must be followed)

Let the semaphore hold an integer value `value`.

- `semaphore_init(sem, initial_value)`:
 - Initializes the semaphore with `initial_value`.
 - Assume `initial_value >= 0`.
- `semaphore_wait(sem)` (P / down):
 - If `value > 0`, decrement `value` by 1 and return.
 - If `value == 0`, the calling thread must wait until `value > 0`, then decrement and return.
 - Waiting must be implemented by looping and calling `sched_yield()` (i.e., no blocking primitives).
- `semaphore_signal(sem)` (V / up):
 - Increment `value` by 1.
 - This must eventually allow waiting thread(s) to complete `semaphore_wait()`.

Notes:

- Correctness is required; **FIFO fairness for waiting threads is not required** (unless explicitly stated elsewhere).
- Assume the semaphore value will not overflow `int`.

Mandatory functions

```
void semaphore_init(semaphore* sem, int initial_value);
/* Initializes the semaphore with an initial value (initial_value >= 0). */

void semaphore_wait(semaphore* sem);    // Decrement (wait / P)
void semaphore_signal(semaphore* sem);  // Increment (signal / V)
```

Required implementation approach

- Protect the semaphore's internal `value` using the **ticket lock from the lecture**.
- You may reuse/copy the lecture ticket-lock implementation in your solution.
- The semaphore's `value` does **not** need to be atomic if you only access it while holding the ticket lock.

Ticket lock from lecture (reference)

```
typedef struct
{
    atomic_int ticket;
    atomic_int cur_ticket;
} ticket_lock;

void ticketlock_init(ticket_lock* lock)
{
    atomic_init(&lock->ticket, 0);
    atomic_init(&lock->cur_ticket, 0);
}

void ticketlock_acquire(ticket_lock* lock)
{
    int my_ticket = atomic_fetch_add(&lock->ticket, 1);

    while (atomic_load(&lock->cur_ticket) != my_ticket)
    {
        sched_yield();
    }
}

void ticketlock_release(ticket_lock* lock)
{
    atomic_fetch_add(&lock->cur_ticket, 1);
}
```

Testing requirements (submit with your solution)

You must submit **three test files** (each file is one test program with `main()`), placed under `PQ1/tests/` (paths below are relative to the `PQ1/` directory):

1. `tests/test_unit.c` — **Unit test (single thread)**: basic `init/signal/wait` behavior.
2. `tests/test_concurrent.c` — **Concurrency test**: create **8 threads** that all call `semaphore_wait()` once; the main thread calls `semaphore_signal()` exactly 8 times; verify all 8 threads finish.
3. `tests/test_stress.c` — **Stress test**: create **16 threads** and run many operations (e.g., each thread performs at least 10,000 `wait / signal` operations in a loop); verify no deadlock and that the final semaphore value is exactly what you expect.

Each test file must:

- return `0` on success
- return `1` on failure
(Printing a short failure message to `stdout / stderr` is recommended.)

Files to submit

- Implementation: `t1_semaphore.c` and `t1_semaphore.h`
- Tests:
`tests/test_unit.c`


```
tests/test_concurrent.c
tests/test_stress.c
```

PQ2 — Condition variable

Implement a condition variable. This is a synchronization primitive that allows threads to wait until a particular condition becomes true. You must implement the functions declared in `cond_var.h`.

Reusing PQ1 code:

Your condition variable uses the ticket lock from PQ1. You **must copy** the relevant files (e.g. `t1_semaphore.c`, `t1_semaphore.h`) **into the `pq2/` directory** and compile them there.

Do **not** include files using paths like `../PQ1/t1_semaphore.h` — that will break compilation because the corresponding `.c` file will not be found by the build script.

Goal

Implement a condition variable that works with an **external ticket lock** (passed as `ext_lock` to `condition_variable_wait`).

You must **not** use any OS/library synchronization primitives in your condition variable implementation (no `pthread_mutex/cond/semaphore/rwlock`, etc.).

When a thread is waiting, it must **not hold** `ext_lock`. Waiting should be implemented by busy-waiting with `sched_yield()` (directly or indirectly via your own primitives).

Mandatory functions

```
void condition_variable_init(condition_variable* cv);
/*
 * Initializes the condition variable pointed to by 'cv'.
 */

void condition_variable_wait(condition_variable* cv, ticket_lock* ext_lock);
/*
 * Precondition: the calling thread holds 'ext_lock'.
 *
 * Causes the calling thread to wait on the condition variable 'cv'.
 * While waiting, the thread must release 'ext_lock' and must reacquire it before
returning.
 *
 * IMPORTANT correctness rules (no "lost notifications"):
 *
 * - Signals are NOT remembered:
 *   If condition_variable_signal/broadcast is called when no threads are waiting,
 *   it has no effect.
 *
 * - A waiting thread must not get "stuck" due to a race:
 *   If a thread T is inside condition_variable_wait (it has started the wait operation
 *   and has not returned yet), and another thread later calls
 *   condition_variable_signal/broadcast, then T must eventually wake and return
 *   (after reacquiring ext_lock).
 */

void condition_variable_signal(condition_variable* cv);
/*
 * Wakes up exactly one thread currently waiting on the condition variable 'cv' (if any).
 * If no threads are waiting, this function does nothing.
 */

void condition_variable_broadcast(condition_variable* cv);
/*
 * Wakes up all threads currently waiting on the condition variable 'cv'.
 * If no threads are waiting, this function does nothing.
 */
```

Required semantics and assumptions

- Assumption: each condition variable is always used with the same external lock `ext_lock` (i.e., do not use the same `cv` with different locks).
- `condition_variable_wait(cv, ext_lock)` :
 - The caller **must hold** `ext_lock` when calling `wait`.
 - The function must:
 - a. Register the thread as "waiting on `cv`" (so a later `signal/broadcast` will not be missed),

- b. Release `ext_lock` ,
- c. Wait until woken by `signal` or `broadcast` ,
- d. Reacquire `ext_lock` ,
- e. Return.

- `signal` wakes **one** currently-waiting thread (choice of which one is up to you; FIFO is **not** required).
- `broadcast` wakes **all** currently-waiting threads.
- You may assume the number of threads is finite and reasonable (e.g., up to a few dozens).
- You may assume no integer overflow in internal counters.

Testing requirements (submit with your solution)

Submit the following **three test files** under `PQ2/tests/` (paths below are relative to the `PQ2/` directory):

1. `tests/test_cv_unit.c` — **Unit test (single thread):**

- `condition_variable_init`
- calling `signal` / `broadcast` when nobody is waiting should not crash/hang

2. `tests/test_cv_signal.c` — **Signal test (8 threads):**

- Create **8** threads that all do: acquire `ext_lock` , then `condition_variable_wait(cv, &ext_lock)` , then record that they were released, then release `ext_lock` and exit
- The main thread should wake them using **exactly 8 calls to `condition_variable_signal`** (one at a time)
- Verify that after the i -th `signal` , the number of released threads is exactly i (i.e., signal wakes **one** waiter)

3. `tests/test_cv_broadcast_stress.c` — **Broadcast stress test (16 threads):**

- Create **16** waiting threads
- Use `condition_variable_broadcast` to wake all of them
- Repeat the “wait then broadcast” cycle many times (e.g., 100–1000 iterations)
- Verify no deadlock and that in each iteration all 16 threads were released

Each test file must be a C program with a `main()` that returns:

- `0` on success
 - `1` on failure
- (Printing a short failure message is recommended.)

Files to submit

- Implementation: `cond_var.c` and `cond_var.h`
- Tests:
 - `tests/test_cv_unit.c`
 - `tests/test_cv_signal.c`
 - `tests/test_cv_broadcast_stress.c`

Submit your implementation in `cond_var.c` and any additional files (if there are any).

PQ3 — Readers–Writer lock (simplified, writer-preference)

Implement a readers–writer lock that allows **multiple concurrent readers** or **one exclusive writer**. You must implement the functions declared in `rw_lock.h`.

Reusing PQ1 and PQ2 code:

Your RW-lock may use your ticket lock, semaphore, and/or condition variable from PQ1 and PQ2. You **must copy** the relevant files (e.g. `tl_semaphore.c`, `tl_semaphore.h`, `cond_var.c`, `cond_var.h`) **into the `pq3/` directory** and compile them there.

Do **not** include files using paths like `../PQ1/tl_semaphore.h` or `../PQ2/cond_var.h` — that will break compilation because the corresponding `.c` files will not be found by the build script.

Goal (what correctness means)

Your RW-lock must satisfy:

- **Mutual exclusion for writers:** if a writer holds the lock, no other writer or reader may hold it.
- **Shared access for readers:** if one or more readers hold the lock, no writer may hold it.
- **Writer starvation prevention (required):** if a writer starts waiting, it must eventually acquire the lock (assuming threads continue to run and every thread that acquires the lock eventually releases it).

Simplified fairness policy (must implement)

Use writer-preference:

- If **any writer is waiting**, then **new readers are not allowed to enter** (they must wait).
- Existing readers that already hold the lock may finish normally.
- This policy prevents writer starvation, but it may starve readers (that's OK in this exercise).

Mandatory functions

```
void rwlock_init(rwlock* lock);
/*
 * Initializes the read-write lock.
 */

void rwlock_acquire_read(rwlock* lock);
/*
 * Acquires the lock for reading.
 */

void rwlock_release_read(rwlock* lock);
/*
 * Releases the lock after reading.
 */

void rwlock_acquire_write(rwlock* lock);
/*
 * Acquires the lock for writing (exclusive access).
 */

void rwlock_release_write(rwlock* lock);
/*
 * Releases the lock after writing.
 */
```

Required assumptions (state explicitly)

- Threads do **not** terminate while holding the RW-lock.
- Any thread that acquires the lock eventually releases it.
- Your implementation must not use pthread synchronization primitives (mutex/cond/semaphore/rwlock/etc.) in the RW-lock implementation.
- You may use your primitives from PQ1/PQ2 (ticket lock, unbounded semaphore, condition variable), and you may use `sched_yield()` .

Hints (how to structure it)

- Maintain counters such as:
 - `active_readers` (number of readers currently holding the lock)
 - `waiting_writers` (number of writers waiting to acquire)
 - `active_writer` (0/1)
- With writer-preference, a reader is allowed in only if:
 - `active_writer == 0 and waiting_writers == 0`

Testing requirements (submit with your solution)

Submit the following **three test files** under `PQ3/tests/` (paths below are relative to the `PQ3/` directory). Each file is a standalone C program with `main()` :

1. `tests/test_rw_readers_parallel.c` — **Readers can run together (8 threads):**
 - Create **8 reader threads** that all acquire the read lock, then stay in the read section for a short time (e.g., a loop), then release.
 - Verify (using an atomic counter) that at some point there were **at least 2 readers inside simultaneously**.
 - Verify no deadlock.
2. `tests/test_rw_writer_exclusive.c` — **Writer exclusivity (1 writer + 8 readers):**
 - Create **1 writer thread** that acquires the write lock and, while holding it, checks that:
 - no readers are inside (a shared atomic `readers_in_cs` must be 0)
 - no other writer is inside
 - In parallel, create **8 reader threads** trying to read repeatedly.
 - Verify the writer never overlaps with any reader/writer in the critical section.
3. `tests/test_rw_writer_preference_stress.c` — **Writer-preference / no writer starvation (16 threads):**
 - Create **12 reader threads** that repeatedly: `acquire_read` → short work → `release_read`.
 - Create **4 writer threads** that repeatedly: `acquire_write` → short work → `release_write`.
 - Run for enough iterations (e.g., 5,000 total acquisitions across writers).
 - Verify every writer thread successfully acquires the write lock many times (e.g., at least 100 acquisitions each), i.e., writers do not starve.

Each test file must:

- return `0` on success
 - return `1` on failure
- (Printing a short failure message is recommended.)

Files to submit

- Implementation: `rw_lock.c` and `rw_lock.h`
- Tests:
 - `tests/test_rw_readers_parallel.c`
 - `tests/test_rw_writer_exclusive.c`
 - `tests/test_rw_writer_preference_stress.c`

Final Notes

- **Read the Instructions Carefully:** Ensure that you adhere to all restrictions and requirements.
- **Testing:** Test your code thoroughly with various inputs to ensure correctness, especially under concurrent execution.
 - **Also test without a debugger** since debuggers may affect synchronization behavior and hide bugs.
 - **Debugging:** Start with a small number of threads (e.g., one, then two, then five) and increase gradually.
 - Use debug printouts to help identify issues, but avoid excessive output in submitted code (especially inside tight spin loops).
Keep test output **short and readable**.
- **Code Quality:** Write clean, well-commented code to help the graders understand your implementation. Comments will also help you understand what you are trying to achieve.

- **Unit Tests:** You are (more than) encouraged to use unit tests to validate your synchronization mechanisms.
- **Grading Reminder:** A happy grader is a merciful grader!

Good Luck! 🍀
